Decision Tree — Complete Notes (Hinglish)

Nov 8, 2026 · @Razeen

Decision Tree ek **if-else rules ki tree structure** hai jo data se automatically rules seekhti hai. Yeh non-linear data ke liye perfect hai aur feature scaling ki zarurat nahi hoti.

1. Tree-Based Methods ka Introduction

Tree-based methods ek aisi ML algorithm family hai jo **if-else rules** ki tree structure use karti hai predictions ke liye. Yeh naturally branching decisions lete hain — bilkul waise jaise hum real life mein soochte hain.

Simple Example: "Kya aaj cricket khelne jaayein?"

- Mausam kaisa hai? → Baarish → NO
- Dhoop hai? → Haan → YES
- Kitne log hain? → Kam → NO

Yahi decision-making process ek tree mein encode hoti hai.

Kyun Use Karte Hain?

Feature	Linear Regression	Decision Tree
Non-linear data	Struggle karta	Handle karta
Feature scaling	Zaruri	Zaruri nahi
Interpretable	Partial	Bahut easy
Missing values	Dikkat	Handle kar sakta
Categorical data	Encode karna padta	Direct use

Tree-based Methods Ki Family

- Decision Tree — single tree, baseline, interpretable
- Random Forest — 100+ trees, better accuracy
- Gradient Boosting — trees in sequence, even better
- XGBoost / LightGBM — production-grade, state-of-the-art

Real Life Applications: Loan approval (banking), disease diagnosis (medical), customer churn (e-commerce), employee attrition (HR).

2. History aur Terminology

Algorithm Ka Safar

Saal	Person / Paper	Contribution
1963	Morgan & Sonquist	AID — pehla tree method
1979	Ross Quinlan	ID3 — Information Gain use kiya
1984	Breiman et al.	CART — Gini impurity introduce ki
1993	Ross Quinlan	C4.5 — ID3 ka improved version
2001	Leo Breiman	Random Forest — multiple trees

Sklearn mein **CART algorithm** use hota hai (Classification and Regression Trees).

Important Terms

Term	Matlab
Root Node	Tree ka starting point, pehla question
Internal Node	Beech ka node, aur questions poochta hai
Leaf Node	Aakhri node, yahaan prediction hoti hai
Branch / Edge	Do nodes ko connect karne wali line
Depth	Root se leaf tak kitne levels hain
Pure Node	Sirf ek class (Gini = 0)
Impure Node	Mixed classes (Gini > 0)
Split	Node ko do parts mein todna
Pruning	Overfitting rokne ke liye branches kaatna
Parent Node	Jis node se split hua
Child Node	Split ke baad bane naye nodes

3. Gini Impurity

Gini Impurity measure karta hai ki ek node kitna **mixed** hai.

- Pure Node (Gini = 0): sirf ek hi class
- Most Impure (Gini = 0.5): 50-50 split (binary case)

Formula

```
\text{Gini}(node) = 1 - \sum_{i=1}^{C} p_i^2
```

Jahaan p_i = uss class ke samples ka fraction.

Examples

Node Contents	p(Yes)	p(No)	Gini Calculation	Gini
[Y, Y, Y, Y]	1.0	0.0	$1 - (1.0^2 + 0.0^2)$	0.0 (Pure)
[Y, Y, Y, N]	0.75	0.25	$1 - (0.75^2 + 0.25^2)$	0.375

Node Contents	p(Yes)	p(No)	Gini Calculation	Gini
[Y, Y, N, N]	0.5	0.5	$1 - (0.5^2 + 0.5^2)$	0.5 (Max Impure)

Gini vs Entropy

	Gini	Entropy
Formula	$1 - \Sigma\, p^2$	$-\Sigma\, p \log_2(p)$
Range	[0, 0.5]	[0, 1]
Speed	Faster (no log)	Slower
Sklearn default	Haan (`criterion='gini'`)	Optional (`criterion='entropy'`)
Results	Usually similar	Usually similar

Intuition: Ek class room mein agar sab bachche same subject pasand karte hain — woh pure (Gini=0) hai. Alag-alag subjects — impure. Tree algorithm hamesha uss split ko choose karta hai jo Gini ko sabse zyada reduce kare.

4. Tree Construction — Gini se Tree Kaise Banta Hai

CART Algorithm Steps

1. Root node pe sare training samples hain
2. Har feature ke har possible threshold pe split try karo
3. Har split ke liye Weighted Gini calculate karo
4. Jo split Gini ko sabse zyada reduce kare → woh choose karo
5. Do child nodes banao
6. Yeh process recursively repeat karo jab tak node pure ho ya stopping criteria hit ho

Weighted Gini Formula

```
\text{Weighted Gini} = \frac{n_{left}}{n} \times Gini_{left} +
\frac{n_{right}}{n} \times Gini_{right}
```

Gini Gain

```
\text{Gini Gain} = Gini_{parent} - \text{Weighted Gini}_{split}
```

Jis split ka **Gini Gain maximum** ho → woh split choose hota hai.

Manual Example

10 customers: 6 ne loan liya (Yes), 4 ne nahi (No). Parent Gini = $1 - (0.6^2 + 0.4^2) = 0.48$

Split	Left Node	Right Node	Weighted Gini	Gini Gain
Age > 30	Gini=0.48 (5 samples)	Gini=0.32 (5 samples)	0.40	0.08
Income > 5L	Gini=0.375 (4 samples)	Gini=0.278 (6 samples)	0.317	0.163

Income > 5L ka Gini Gain zyada hai → yeh root split choose hoga.

Stopping Criteria

Parameter	Matlab
max_depth	Tree kitna deep ja sakta hai
min_samples_split	Split ke liye minimum samples
min_samples_leaf	Leaf mein minimum samples
ccp_alpha	Cost-Complexity Pruning

In conditions mein se koi bhi hit hone pe tree grow karna band kar deta hai.

5. Implementation — sklearn mein Decision Tree

Old Method vs New Method

	Old (Scratch)	New (sklearn)
Gini calculation	Manual loop	Automatic
Splitting	Manual recursion	Automatic
Pruning	Manual	max_depth, min_samples_*
Visualization	Custom code	plot_tree()

	Old (Scratch)	New (sklearn)
Production use	Nahi	Haan

Basic Usage

```
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.model_selection import train_test_split, cross_val_score

# Model banana
dt = DecisionTreeClassifier(
    criterion='gini',       # 'gini' ya 'entropy'
    max_depth=5,            # Kitna deep tree ho
    min_samples_split=2,    # Split ke liye min samples
    min_samples_leaf=1,     # Leaf mein min samples
    random_state=42
)
dt.fit(X_train, y_train)

# Predict karo
y_pred = dt.predict(X_test)

# Tree visualize karo
plot_tree(dt, feature_names=feature_names,
          class_names=class_names, filled=True)

# Text rules nikalo
print(export_text(dt, feature_names=feature_names))
```

Important Parameters

Parameter	Default	Purpose
criterion	'gini'	Impurity measure
max_depth	None	Tree ki maximum depth
min_samples_split	2	Node split ke liye min samples
min_samples_leaf	1	Leaf mein min samples
max_features	None	Har split pe features consider karo
random_state	None	Reproducibility ke liye

Parameter	Default	Purpose
ccp_alpha	0.0	Cost-Complexity Pruning

Useful Methods

- `dt.get_depth()` — actual tree depth
- `dt.get_n_leaves()` — leaf nodes ki count
- `dt.feature_importances_` — feature importance array
- `dt.score(X_test, y_test)` — accuracy
- `dt.predict_proba(X)` — class probabilities

6. Overfitting aur Pruning

Problem: Unlimited depth tree training data 100% memorize kar leta hai — naye data pe poorly perform karta hai.

Bias-Variance Tradeoff Trees Mein

max_depth	Behavior	Problem
Bahut kam (1–2)	Underfitting	High bias, low variance
Sweet spot (3–7)	Good balance	Best generalization
Bahut zyada (None)	Overfitting	Low bias, high variance

Sweet Spot Kaise Dhundhen?

```
depths = range(1, 20)
cv_scores = []
for d in depths:
    dt = DecisionTreeClassifier(max_depth=d, random_state=42)
    cv = cross_val_score(dt, X, y, cv=5).mean()
    cv_scores.append(cv)

best_depth = depths[np.argmax(cv_scores)]
print(f"Best depth: {best_depth}")
```

Pruning ke Types

- **Pre-pruning (Early Stopping):** Tree grow karte waqt hi rok do — `max_depth`, `min_samples_split`, `min_samples_leaf`
- **Post-pruning:** Poora tree banao, phir unnecessary branches kaato — `ccp_alpha` (Cost-Complexity Pruning)

GridSearchCV se Best Parameters

```
param_grid = {
    'max_depth': [3, 5, 7, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 5],
    'criterion': ['gini', 'entropy']
}
gs = GridSearchCV(DecisionTreeClassifier(random_state=42),
                  param_grid, cv=5, scoring='accuracy', n_jobs=-1)
gs.fit(X_train, y_train)
print(gs.best_params_)
```

Rule of Thumb: Hamesha `cross_val_score` se evaluate karo, sirf test accuracy pe mat jaao.

7. Feature Importance aur Interpretation

Feature Importance Kya Hai?

Har feature ka score batata hai ki usne total Gini reduction mein kitna contribute kiya. Root split ke feature ka importance usually highest hota hai.

```
importances = pd.Series(
    dt.feature_importances_,
    index=feature_names
).sort_values(ascending=False)
print(importances)

# Bar chart
importances.plot(kind='barh')
```

Tree ko Explain Karna — Real World Mein

Decision Tree ka sabse bada advantage yeh hai ki koi bhi samajh sakta hai:

```
export_text(dt, feature_names=features)
# Output:
# |--- petal length <= 2.45
# |   |--- class: setosa
# |--- petal length > 2.45
# |   |--- petal width <= 1.75
# |   |   |--- class: versicolor
# |   |--- petal width > 1.75
# |   |   |--- class: virginica
```

Yeh rules ek doctor ko, bank manager ko, ya HR executive ko explain kar sakte hain bina kisi technical knowledge ke.

Single Prediction Explain Karo

```
# Ek patient ke liye predict karo
patient = X_test.iloc[0]
pred = dt.predict([patient])[0]
prob = dt.predict_proba([patient])[0]
print(f"Prediction: {pred}")
print(f"Probability: {prob}")
# Node path trace karne ke liye:
node_indicator = dt.decision_path([patient])
```

False Negatives Medical Mein Zyada Dangerous Hain

Medical diagnosis mein:

- False Negative (bimari miss karna) = Dangerous — treatment nahi milega
- False Positive (healthy ko flag karna) = Extra tests, cost zyada

Isliye `classification_report` mein Recall (Sensitivity) pe dhyan do, sirf accuracy pe nahi.

8. Summary aur Next Steps

Poore Topic Ka Ek Nazar Mein

Topic	Key Takeaway
Tree-based Methods	Non-linear data ke liye perfect, interpretable

Topic	Key Takeaway
Gini Impurity	Node ki purity measure karta hai (0 = pure, 0.5 = max impure)
Tree Construction	Greedy algorithm — har step pe best Gini Gain split choose
Implementation	DecisionTreeClassifier, max_depth sabse important parameter
Overfitting	Deep tree = overfit; cross-val se sweet spot dhundho
Feature Importance	Root split ka feature usually most important
Interpretation	export_text se rules nikalo, kisi ko bhi explain kar sakte hain

Decision Tree Ke Limitations

- High variance — thoda sa data change aur poora tree change
- Thoda non-robust — outliers se affect hota hai
- Axis-aligned splits only — diagonal boundaries properly capture nahi kar sakta

Aage Kya Seekhein?

Algorithm	Kaise Improve Karta Hai	Kab Use Karo
Random Forest	100+ trees average — variance kam	General purpose, strong baseline
Gradient Boosting	Errors pe focus karta hai sequentially	Better accuracy chahiye
XGBoost / LightGBM	Optimized boosting	Production, competitions

Decision Tree ek achha **interpretable baseline** deta hai. Agar accuracy enough nahi toh Random Forest try karo. Production mein mostly XGBoost / LightGBM best perform karta hai tabular data pe.